How-To Library

Informatica

Invoking Test Data Management REST APIs Using cURL

© Copyright Informatica LLC 2019, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

You can use Test Data Management REST APIs to automate tasks that you perform in Test Data Management (TDM). This article describes how you can use cURL to run TDM REST APIs. cURL is a command line tool and library for transferring data with URLs. It is used in command lines or scripts to transfer data.

Supported Versions

- Test Data Management 10.2.0 HotFix 2

Table of Contents

Overview. . . . 2
Example. . . . 3
Prerequisites. . . . 4
Rules and Guidelines. . . . 4
Generate BasicAuth Authentication Token. . . . 5
Create and Run the Required REST API Requests 6
API Reference. . . . 7
Identify the Project ID. . . . 9
Identify the Connection ID. . . . 9
View a List of Schemas in the Connection. . . . 10
View Data Sources in the Schema. . . . 11
Import Source Metadata into the Project. . . . 11
View Rules in the Project to Identify the Rule ID. . . . 12
View Columns to identify the Column ID. . . . 13
Assign a Masking Rule to a Column. . . . 14
View Plans. . . . 14
Generate a Workflow. . . . 14
Monitor the Workflow Generation Job. . . . 15
Run the workflow. . . . 15
Monitor the Workflow Execution Job. . . . 16

Overview

A REST API is an application program interface that uses HTTP or HTTPS requests to perform tasks.

TDM REST APIs are intended for TDM developer users who want to automate and run TDM tasks. You can use TDM REST APIs to perform Test Data Manager, test data warehouse, and the self-service portal tasks.

The APIs use the GET, POST, PATCH and DELETE methods to perform tasks.

Note: You cannot use Test Data Management REST APIs in a Kerberos environment.

Use the Test Data Management REST API to perform the following tasks in Test Data Management:

Projects

You can search for projects. You can assign and unassign rules on ports and export and import data domain assignments. You can import metadata from a Test Data Manager connection or a PowerCenter repository folder.

Connections

You can create, delete, or import a connection into Test Data Manager. You can view a list of connections. You can view schema and sources in a connection.

Plans

You can search for plans based on projects and plan IDs.

Test Data Manager Jobs

You can search for jobs based on job ID and generate and run jobs. You can create and run workflows.

Use the test data warehouse REST API to perform the following test data warehouse tasks:

Test data warehouse Jobs

You can run data set reset and restore jobs and view the status and information of jobs based on job IDs.

Data sets

You can view a list of all data sets and all versions of a data set.

You can use the self-service portal REST API to perform the following tasks in the self-service portal:

Data packs

You can view a list of all data packs. You can also export the data in a data pack as a CSV file.

Self-service portal Jobs

You can search for and view the status of jobs and run jobs to reset and restore a data pack.

For more information about TDM REST APIs and the tasks that you can perform using the APIs, see the *Test Data Management 10.2.0 HotFix 2 REST API Reference*.

Example

You work with a test team that uses Test Data Management to provision test data for multiple teams within the organization.

Your job is to run plans in Test Data Manager that create test data in the target test environments. Because you need to run similar tasks multiple times, you use the REST APIs to automate the task of assigning rules and running plans to create test data.

You create API requests with cURL scripts to perform required tasks in Test Data Manager. You retrieve the required project, import the metadata, and assign masking rules to required ports. You then identify the required plan and generate and run a workflow to create the test data. You monitor the status of each job to ensure the job finishes successfully.

You can then create a script that runs the APIs to automate the task.

The security domain is Native and the user name/password is Administrator/Administrator in all tasks in this document.

The tasks use the following objects:

- Project: Project_RestAPI01
- Connection: DB01
- Schema: DB01

- Source: BONUS
- Column: ENAME
- Masking rule: Full Name Rule

Perform the following tasks to create and run TDM REST APIs with cURL:

1. Identify the project ID.
2. Identify the connection ID.
3. View the schema in the connection that you want to use.
4. View the data sources in the required schema.
5. Import the source metadata into the project.
6. Identify the masking rule ID.
7. Identify the column ID of the column you want to mask.
8. Assign the masking rule to the column.
9. View the plans in the project to identify the required plan ID.
10. Generate a workflow to mask the data.
11. Monitor the status of the job to ensure the workflow is created successfully.
12. Run the workflow to create the masked test data in the target.
13. Monitor the status of the job to verify that it finishes successfully.

Prerequisites

Before you use TDM REST APIs to automate and perform tasks in Test Data Manager, perform the following tasks:

- Verify that the Test Data Management version includes REST APIs.
- Verify that you have installed cURL and can run cURL commands from the command line.
  cURL is a free and open source software that you can download from the official website.
- Generate a BasicAuth authentication token.

Rules and Guidelines

Read the following rules and guidelines before you use TDM REST APIs:

- You can run cURL commands from any folder location on the Test Data Management machine.
- Informatica recommends that you use Swagger to generate cURL commands. The TDM installer includes Swagger. When you run a REST API request in Swagger, Swagger creates and displays the cURL request. You can save this cURL request to reuse from the command line.
- When you provide the login information to log in from the Swagger user interface, it encodes the information and adds it in each of the API requests that you run during the Swagger session.
- After you generate cURL requests for all tasks that you want to automate, you can create a batch or shell script to run multiple APIs. The script that you use might differ based on the environment and the APIs that you want to run. The article does not include this information.

Generate BasicAuth Authentication Token

TDM REST API uses BasicAuth for authorization. Basic authentication is a simple authentication scheme built into the HTTP protocol. The client sends HTTP requests with the Authorization header that contains the word Basic followed by a space and a base64-encoded string `security domain/username:password`.

You can log in from Swagger or use any Java or other utility to encode the user name and password. This task documents the steps to use Swagger to extract the encoded authorization information. To log in using Swagger, you must enter a valid TDM user name and password.

1. In the address bar of a web browser, enter the URL in the following format:

   `http(s)://<host name>:<port number>/api`

   Where:

   - `<host name>` is the host name or IP address of the machine where you installed TDM.
   - `<port number>` is the port number. The default is 6643 if Transport Layer Security is enabled. The default is 6605 if Transport Layer Security is not enabled.
2. Click Authorize to open the Authorization window.

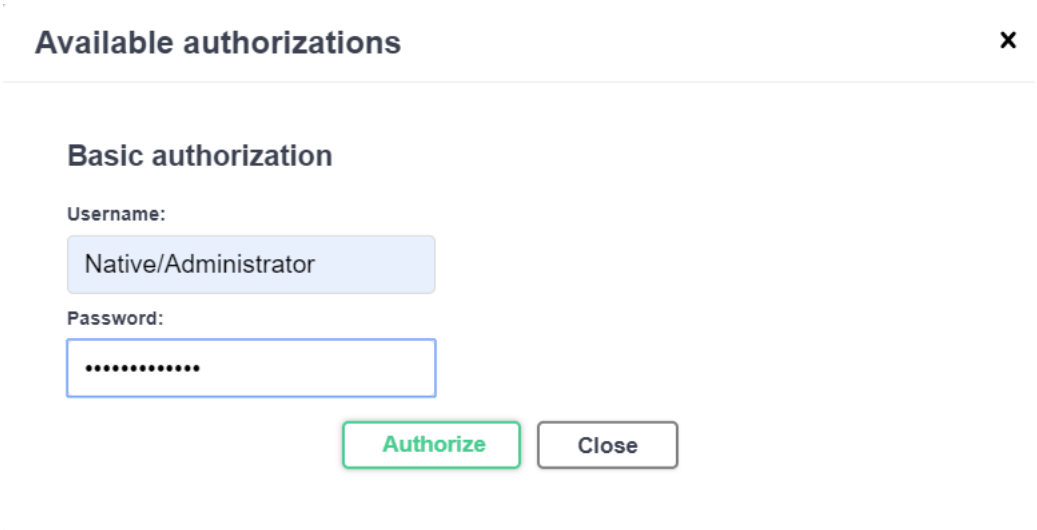
   The Authorize button is on the upper right-hand corner of the page.
3. Enter a user name in the following format:

   `<Security domain>/<user name>`

   Where:

   - `<Security domain>` is the name of the security domain to which the user belongs.
   - `<user name>` is the domain user name.

   The following image shows the Authorization dialog box before you click Authorize:

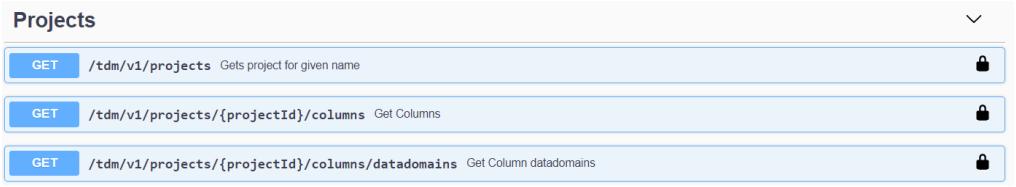


4. Click **Authorize**.

   Swagger appends the encoded user name and password to all REST API requests that you run from Swagger.

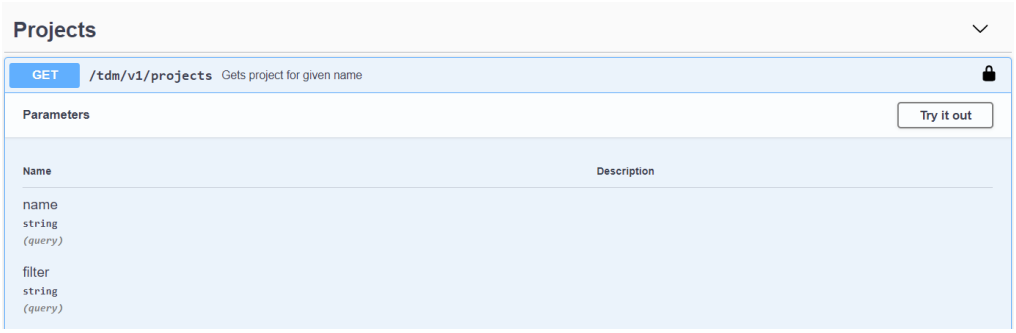
5. You can run an API request in Swagger to identify the encoded authorization information.
   a. For example, run the GET `/tdm/v1/projects` API. Click the row to open the corresponding API section.

      The following image shows the APIs available under Projects:

   b. Click the **Try it out** button to view the parameters and value fields.

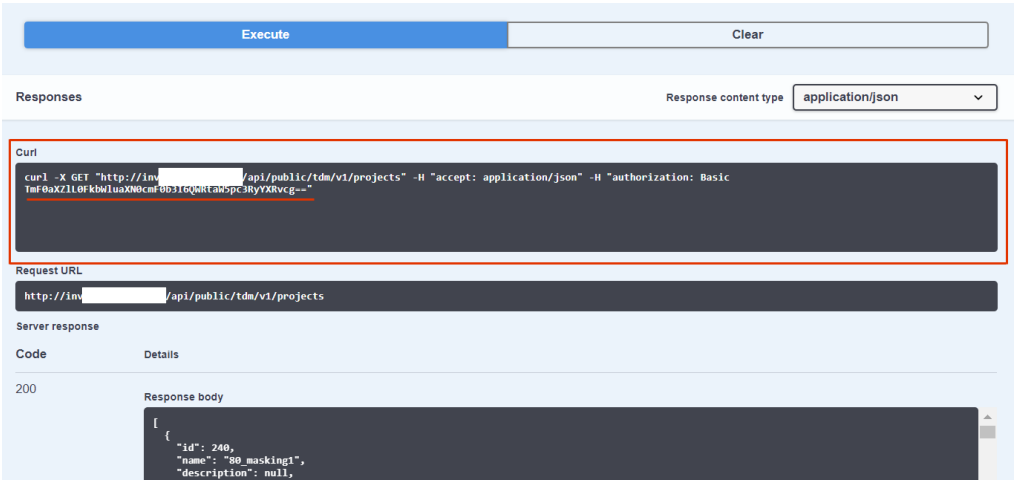
      The following image shows the expanded /tdm/v1/projects API view:

      You can run the `projects` API without adding filters.

   c. Click **Execute** to run the API.

      The following image shows the Swagger page with the responses:

      You see the cURL request that you can save and reuse. The request contains the encoded user authorization details that you can reuse.

Create and Run the Required REST API Requests

Create the API requests and run the REST APIs to perform tasks in Test Data Manager.

To reuse API requests and run from the command line, use Swagger to generate cURL requests. You can also create the requests manually and use them to run the REST APIs from the command line.

This article uses sample REST API requests to perform specific tasks. You must replace the values if you reuse the requests to perform tasks in your Test Data Management setup.

API Reference

The following table lists properties that you use in each REST API request in this document:

REST API	Method	Properties
/tdm/v1/projects	GET	The API request uses the following property: - name. Name of the project. The example uses the value `Project_RestAPI01`.
/tdm/v1/connections	GET	The API request uses the following property: - filter. Filters based on criteria. The example uses the name of the connection DB01 as the filter.
/tdm/v1/connections/{connectionId}/schemas	GET	The API request uses the following property: - connectionId. Required. ID of the connection that contains the schemas. The example uses the ID of the DB01 connection: `U:1kjBTQOEemYiOETycM8Kg`
/tdm/v1/connections/{connectionId}/schemas/{schemaname}/sources	GET	The API request uses the following properties: - connectionId. Required. ID of the connection that contains the schemas. The example uses the ID of the DB01 connection: `U:1kjBTQOEemYiOETycM8Kg`. - schemaName. Name of the schema that contains the sources. The example uses the `DB01` schema. - filter. Filters based on criteria. The example uses the schema name as the filter.
/tdm/v1/projects/{projectId}/metadata/connection	POST	The API request uses the following properties: - projectId. ID of the project into which you want to import the metadata. The example uses the value `Project_RestAPI01`. - schedule. Default is 0 and runs the job immediately. The example uses 0. - datasources. For each connection from which you want to import metadata, enter the following details: - connectionId. ID of the TDM connection from which you want to import metadata. The example uses the ID of the DB01 connection: `U:1kjBTQOEemYiOETycM8Kg`. - name. Schema name. The example uses `DB01`. - sources. For each source that you want to import enter the following: - name. Name of the table. Th example uses `EMP`.
/tdm/v1/projects/{projectId}/rules	GET	The API request uses the following property: - projectId. ID of the project that you are working in and that contains the rules. The example uses the value `Project_RestAPI01`.

REST API	Method	Properties
/tdm/v1/projects/{projectId}/columns	GET	The API request uses the following property: - projectId. ID of the project that you are working in and that contains the columns. The example uses the value `Project_RestAPI01`.
/tdm/v1/projects/{projectId}/columns/ruleassignments	POST	The API request uses the following properties: - projectId. ID of the project that you are working in and that contains the rules. The example uses the value `Project_RestAPI01`. - id. Existing rule assignment ID. The example uses the value 0 as there is no previous rule assignment. - port_id. Required. Column ID number. The example uses the column ID 79395. - name. Column name. The example uses the value ENAME. - rule_id. Required. Rule ID number. The example uses the value 37. - policy. Policy ID number. The example uses the value 0 as the rule is an additional masking rule in the project. - params. Parameter. This is not used in the example.
/tdm/v1/projects/{projectID}/plans	GET	The API request uses the following property: - projectId. ID of the project that you are working in and that contains the plans. The example uses the value `Project_RestAPI01`.
/tdm/v1/jobs	POST	The API request uses the following properties: - Content-Type. Format to submit forms that can contain files. The example uses `multipart/form-data`. - type. Generate or Execute. In the example, we run the API twice. We first generate the workflow and then execute the workflow. - projectId. ID of the project that you are working in and that contains the plan you want to run. The example uses the value `Project_RestAPI01`. - planId. Required. ID of the plan for which you want to generate or run a workflow. The example uses the value `2211`. - integrationServiceName. Name of the Integration Service to run the job. The example uses the value `PCIS`.
/tdm/v1/jobs/{jobId}	GET	The API request uses the following property: - jobId. Required. ID of the job that you want to monitor. In the example, we run the API twice. We first monitor the workflow generation job with job ID `3418` and then monitor the workflow execution job with job ID `3420`.

Identify the Project ID

To perform tasks on objects in a project, you include the project ID in the request.

You can run the `/tdm/v1/projects` API to identify the project ID.

1. Open a command prompt window from the required directory location and enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/projects?name=Project_RestAPI01" -H
"accept: application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response to identify the project ID.

   The following code is the response that the sample request returns:

```
[
{
    "id":2412,
    "name":"Project_RestAPI01",
    "description":"Rest API test",
    "created_by_name":"Administrator",
    "creation_date":1561615732882,
    "creation_date_str":"06/27/2019 11:38:52 AM",
    "last_updated_date":1561615733321,
    "last_updated_date_str":"06/27/2019 11:38:53 AM",
    "last_updated_by_name":"Administrator",
    "status":"Updated",
    "folder_name":"Project_RestAPI01",
    "repository_name":"PCREPO",
    "owner":"Administrator",
    "owner_namespace":"Native"
}
]
```

If you do not enter the name of a project as part of the request, the response includes information on all projects. From the response, you identify the project ID is 2412.

Identify the Connection ID

To use a connection in a REST API request, you include the connection ID.

You can run the `/tdm/v1/connections` API to identify the connection ID. Use the connection name as a filter to ensure that the response includes only the information that you require.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/connections?filter=DB01" -H "accept:
application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response to identify the connection ID.
   The following code is the response that the sample request returns:

```
[
{
"id":"U:1kjBTQOEemYiOETycM8Kg",
"name":"DB01",
"connection_type":"Relational",
"db_type":"Oracle",
"description":"",
"connection_properties":
[{"name":"parallel_mode",
"value":"false",
```

```
"comment":null},
{"name":"owner_domain","value":"Domain","comment":null},
{"name":"owner_username","value":"Administrator","comment":null},
{"name":"jdbc_url","value":"jdbc:informatica:oracle://localhost:
1521;SID=orcl","comment":null},
{"name":"use_kerberos_authentication","value":"false","comment":null},
{"name":"connection_string","value":"orcl","comment":null},
{"name":"code_page","value":"7-bit ASCII","comment":null},
{"name":"password","value":"*****","comment":null},
{"name":"owner_namespace","value":"Native","comment":null},
{"name":"username","value":"DB01","comment":null},
{"name":"passthrough_security_enabled","value":"false","comment":null},
{"name":"connection_retry_period","value":"0","comment":null},
{"name":"use_parameter_in_password","value":"false","comment":null}]
}
]
```

You identify the connection ID for the DB01 connection is: U:1kjBTQOEemYiOETycM8Kg

View a List of Schemas in the Connection

You can run a REST API to view a list of schemas in the connection from which you can import the source metadata.

You can run the `/tdm/v1/connections/{connectionId}/schemas` API to view a list of schemas.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/connections/U:1kjBTQOEemYiOETycM8Kg/
schemas" -H "accept: application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.

3. Review the response to view a list of schemas.
   The following code is the response that the sample request returns:

```
[
"ANONYMOUS","DB01","DB02","DBSNMP",
"DIP","DIS101","DOMAIN102",
"DOMAIN102HF1","DOMAIN102R1","DOMAIN961HF3",
"ECOM","EXFSYS","FLOWS_FILES","GCS_02509644",
"HR","IX","JUNIT","JUNITS101",
"KUMARPRO","KUMARTWO","MART102",
"MART102_NEW","MART102HF1","MART500",
"MART_TEST_01","MARTJUNITS101","MDDATA",
"MDSYS","MGMT_VIEW","MMS101","MRS101","MRS102",
"MRS102HF1","MRS102R1","MRS961","MRS961HF3",
"MRS961HF4","OE","OLAPSYS","ORACLE_OCM","ORDDATA",
"ORDPLUGINS","ORDSYS","OUTLN","OWBSYS","OWBSYS_AUDIT",
"PC101","PC961HF3","PC961HF4","PCADMIN","PCDIS","PCMRS",
"PCREPO","PCRS101","PCRS102","PCRS102HF1","PCRS102R1",
"PCRS961","PCRS961EBF","PCRS961HF3","PCRS961HF3TEST",
"PCRS961HF4","PM","PROF971PVT","PROFILE102R1","PROFILING102",
"REPOTDW102","RESMED971HF1","SCOTT","SH","SI_INFORMTN_SCHEMA",
"SPATIAL_CSW_ADMIN_USR","SPATIAL_WFS_ADMIN_USR","STORAGE102",
"STORAGE102R1","SYS","SYSMAN","SYSTEM","TDM101","TDM101_UPGRADE",
"TDM102","TDM102_JUNITS","TDM102_R1","TDM102HF1",
"TDM102R1","TDM102R1EBF","TDM961","TDM961PVT2",
"TDM961PVT3","TDM970","TDM971","TDM971HF1",
"TDM971MART02","TDM971PVT","TDM971PVT1",
"TDM_REPO","TDM_REPO_02","TDW","TDW101",
"TDW102HF1","TDW102HF1_02","TDW971MART",
"TDW971PVT","TDW971REPO","TDW_02","TDW_MART",
"TDW_REPO","TDW_REPO_02","TDW_REPO_2",
"TDW_REPO_TEST_01","TDWMART","TDWMART101","TDWREPO",
"TDWREPO102","TDWREPO102_TEMP","TDWREPO500",
"TDWRESET101","TDWTARGET102","TDWTEST02",
"user","VERIZON02283505","WMSYS","XDB","XS$NULL"
]
```

You identify the schema DB01 that you want to use from the list of schemas.

View Data Sources in the Schema

You can run a REST API to view a list of data sources in the schema you identify.

You can run the `/tdm/v1/connections/{connectionId}/schemas/{schemaname}/sources` API to view a list of data sources in the schema.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/connections/U:1kjBTQOEemYiOETycM8Kg/
schemas/DB01/sources?filter=%7B%22name%22%3A%20%22DB01%22%7D" -H "accept: application/
json" -H "authorization: Basic TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

   The request includes the URL encoded form of the filter `={"name": "DB01"}`. The filter used is the schema name DB01.

2. Press Enter.

3. Review the response to view a list of sources in the schema.
   The following code is the response that the sample request returns:

```
[
{"name":" -depart ment -"},{"name":"-EmplOYee- 1 "},{"name":"."},
{"name":".spl/,_@"},{"name":"ALL_SUPPORTED01"},{"name":"ALL_TYPES"},
{"name":"BONUS"},{"name":"CDS_MDTL_MEMB_DTL"},{"name":"DATATYPES_DBCLOB_BLOB"},
{"name":"DATE_TIME_TYPES"},{"name":"DATE_TIME_TYPES_02"},
{"name":"DATE_TIME_TYPES_03"},{"name":"DATE_TIME_TYPES_04"},
{"name":"DEPT"},{"name":"DIAMOND_A1"},{"name":"DIAMOND_B1"},
{"name":"DIAMOND_C1"},{"name":"DIAMOND_D1"},{"name":"DIAMOND_E1"},
{"name":"DT_TYPES_TIMESTAMP"},{"name":"DT_TYPES_TZ"},
{"name":"DT_TYPES_TZ6"},{"name":"DT_TYPES_TZ_LOCAL"},
{"name":"D_STRING1"},{"name":"EMP"},{"name":"EMPLOYEE"},
{"name":"EMPLOYEE_TDW"},{"name":"ENT_TEST"},
{"name":"EXPORT_JOB_SQLDEV_1809"},{"name":"EXPORT_JOB_SQLDEV_1810"},
{"name":"IMPORT_JOB_SQLDEV_1977"},{"name":"IMPORT_JOB_SQLDEV_1983"},
{"name":"LOBS"},{"name":"LOBSTAB"},{"name":"SALGRADE"},
{"name":"TDM_2642"},{"name":"TDM_2642_2"},{"name":"TDM_2642_3"},
{"name":"TDM_2642_4"},{"name":"TDM_2642_5"},{"name":"TDM_2642_6"},
{"name":"TDW_TEST"},{"name":"TEMP"},{"name":"TEMP01"},
{"name":"TEMP02"},{"name":"TEMP_TWO_VARCHARS"},
{"name":"TESTDATA_RESTART"},{"name":"TEST_ALL_TYPES"},
{"name":"TEST_DEPT"},{"name":"TEST_EMP"},{"name":"TF2P2"},
{"name":"TI482"},{"name":"TRAF23"},{"name":"TYPES_CHAR"},
{"name":"TYPES_DATES"},{"name":"TYPES_DATES_NEW"},
{"name":"TYPES_INTERVAL"},{"name":"TYPES_LOBS"},{"name":"TYPES_NUMBER"},
{"name":"TYPES_VARCHAR"},{"name":"T_CUSTOMER"},{"name":"T_DEPT"},{"name":"T_DEPT_ZB"},
{"name":"T_EMP"},{"name":"T_EMP_ZB"},{"name":"T_PRODUCT"},
{"name":"T_SALE"},{"name":"a/cCounT4,_ @$"},
{"name":"special/.,_@"},{"name":"spl/,_@"},
{"name":"spl/,_@'"},{"name":"spl/,_@A"}
]
}
]
```

You identify the data source BONUS that you want to use from the list of schemas.

Import Source Metadata into the Project

To use the required source metadata, you must import the source metadata into the project.

You can run the `/tdm/v1/projects/{projectId}/metadata/connection` API to import the source metadata from the required connection.

1. Enter the following cURL API request:

```
curl -X POST "http://<host>:<port>/api/public/tdm/v1/projects/2412/metdata/connection" -
H "accept: application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg==" -H "Content-Type: application/json" -d
```

```
"{ \"connectionId\": \"U:-1kjBTQOEemYiOETycM8Kg\", \"datasources\": [ { \"name\":
\"DB01\", \"sources\": [ { \"name\": \"BONUS\" } ] } ], \"schedule\": 0}"
```

2. Press Enter.
3. Review the response to verify the job status.
   The following code is the response that the sample request returns:

```
{"id":3217,"name":"Import from Source","description":"Connection: DB01","status":"In
Queue","start_date":0,"end_date":0,"job_type":"Import from Ssource",
"created_by_name":"Native/Administrator",
"creation_date":1561618571574,
"creation_date_str":"06/27/2019 12:26:11 PM",
"last_updated_date":1561618571574,
"last_updated_date_str":"06/27/2019 12:26:11 PM",
"last_updated_by_name":"Native/Administrator",
"schedule_on":0,
"paused_at_step":0,
"percent_complete":0,
"job_action_type":null,
"child_job_id":-1}
```

The BONUS source metadata from DB01 schema in the DB01 connection is imported into the `Project_RestAPI01` project.

View Rules in the Project to Identify the Rule ID

You need to assign a masking rule to a column before you can run the plan to create the test data.

You require the rule ID to use REST APIs to assign a rule to a column.

You can run the `/tdm/v1/projects/{projectId}/rules` API to view a list of policies and rules in the project.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/projects/2412/rules" -H "accept:
application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response to identify the rule ID.
   The following code is the response that the sample request returns:

```
{"projPolicies":[],"prefGenRules":[],"projRules":[],
"additional_rules":
[
{"id":37,"version":1,
"name":"Full Name Rule",
"description":"Rule to mask Full Name",
"overideAble":false,
"active":true,
"type":"rule",
"instruction":null,"domains":null,
"projects":null,
"masking_type":"SUBSTITUTION",
"data_type":"String",
"created_by":1,
"creation_date":1553495089611,
"creation_date_str":"03/25/2019 11:54:49 AM",
"last_updated_by":1,
"last_updated_date":1556200406418,
"last_updated_date_str":"04/25/2019 07:23:26 PM",
"associated_data_domains":null,
"masking_mapplet":null,
"rule_parameters":null,
"rule_ports":null,
```

```
"associated_rule_port":null,
"rule_assignments":null,
"rule_assignment_groups":null,
"param_map":{},
"rule_type":"STANDARD"}
],
"gen_rules":[]}
```

You view a list of all the rules and policies in the Project_RestAPI01 project. From the list you identify the rule that you want to use in your plan and the rule ID: `Full Name Rule` with rule ID 37.

View Columns to identify the Column ID

Identify the column to which you want to assign the masking rule.

You require the column ID to use REST APIs to assign a rule to a column.

You can run the `/tdm/v1/projects/{projectId}/columns` API to view a list of columns in the project.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/projects/2412/columns" -H "accept:
application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response to identify the connection ID.
   The following code is the response that the sample request returns:

```
[
{"ds_name":"DB01","source":"BONUS","owner":"DB01","name":"COMM","version":0,"id":
79398,"data_type":"number",
"sensitive":false,"data_domain":"","restricted":false,"constraint_keys":null,"readonly":f
alse,"policy":null,"rule":null,
"rule_id":0,"details":null,"override":null,"assignment_id":null},
{"ds_name":"DB01","source":"BONUS","owner":"DB01","name":"ENAME","version":0,"id":
79395,"data_type":"varchar2",
"sensitive":false,"data_domain":"","restricted":false,"constraint_keys":null,"readonly":f
alse,"policy":null,"rule":null,
"rule_id":0,"details":null,"override":null,"assignment_id":null},
{"ds_name":"DB01","source":"BONUS","owner":"DB01","name":"JOB","version":0,"id":
79396,"data_type":"varchar2",
"sensitive":false,"data_domain":"","restricted":false,"constraint_keys":null,"readonly":f
alse,"policy":null,"rule":null,
"rule_id":0,"details":null,"override":null,"assignment_id":null},
{"ds_name":"DB01","source":"BONUS","owner":"DB01","name":"SAL","version":0,"id":
79397,"data_type":"number",
"sensitive":false,"data_domain":"","restricted":false,"constraint_keys":null,"readonly":f
alse,"policy":null,"rule":null,
"rule_id":0,"details":null,"override":null,"assignment_id":null}
]
```

You view a list of all the columns in the Project_RestAPI01 project. From the list you identify the column that you want to mask in your plan and the column ID: `ENAME` with column ID 79395.

Assign a Masking Rule to a Column

Assign the masking rule to the required column to mask the data in the column.

You can run the `/tdm/v1/projects/{projectId}/columns/ruleassignments` API to assign a masking rule to a column.

1. Enter the following cURL API request:

```
curl -X POST "http://<host>:<port>/api/public/tdm/v1/projects/2412/columns/
ruleassignments" -H "accept: application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg==" -H "Content-Type: application/json" -d
"{ \"id\": 0, \"port_id\": 79395, \"name\": \"string\", \"rule_id\": 37, \"policy\": 0,
\"params\": \"string\"}"
```

2. Press Enter.
3. Review the response.
   The following code is the response that the sample request returns:

```
{"id":1609,"name":"ENAME","policy":201,"params":null,"port_id":79395,"rule_id":37}
```

The response indicates that the masking rule with ID 37 is assigned to the column with ID 79395. The masking assignment ID is 1609.

View Plans

To identify the plan that you want to run, view a list of plans in the project.

You can run the `/tdm/v1/projects/{projectID}/plans` API to identify the plan ID.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/projects/2412/plans" -H "accept:
application/json" -H "authorization: Basic
TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response to identify the plan ID.

   The following code is the response that the sample request of step 1 returns:

```
[
{"id":2211,"name":"Plan_mask_test",
"description":null,"created_by_name":"Administrator",
"creation_date":1561626239405,"creation_date_str":"06/27/2019 02:33:59 PM",
"last_updated_date":1561626239405,"last_updated_date_str":"06/27/2019 02:33:59 PM",
"last_updated_by_name":"Administrator",
"status_type":"Created"}
]
```

You identify the plan ID for the `Plan_mask_test` plan is 2211.

Generate a Workflow

Create a workflow to mask the data.

You can run the `/tdm/v1/jobs` API to generate a workflow.

1. Enter the following cURL API request:

```
curl -X POST "http://<host>:<port>/api/public/tdm/v1/jobs" -H "accept: application/json"
-H "authorization: Basic TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg==" -H "Content-
Type: multipart/form-data" -F "type=Generate" -F "projectId=2412" -F "planId=2211" -F
"integrationServiceName=PCIS"
```

2. Press Enter.

3. Review the response.
   The following code is the response that the sample request returns:

```
{"id":3418,"name":"Generate Workflow","description":"Plan: Plan_mask_test",
"status":"In Queue","start_date":0,"end_date":0,
"job_type":"Generate Workflow","created_by_name":null,
"creation_date":1561633089932,"creation_date_str":"06/27/2019 04:28:09 PM",
"last_updated_date":1561633089932,"last_updated_date_str":"06/27/2019 04:28:09 PM",
"last_updated_by_name":null,"schedule_on":0,"paused_at_step":0,"percent_complete":0,
"job_action_type":null,"child_job_id":-1}
```

The job to generate the workflow has an ID of 3418. To verify that the workflow is generated successfully, you can monitor the job.

Monitor the Workflow Generation Job

You can monitor the status of the workflow generation job to confirm that it finishes successfully.

You can run the `/tdm/v1/jobs/{jobId}` API to view the status of a job.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/jobs/3418" -H "accept: application/
json" -H "authorization: Basic TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response.
   The following code is the response that the sample request returns:

```
{"id":3418,"name":"Generate Workflow","description":"Plan: Plan_mask_test",
"status":"In Queue","start_date":0,"end_date":0,
"job_type":"Generate Workflow","created_by_name":null,
"creation_date":1561633089932,"creation_date_str":"06/27/2019 04:15:09 PM",
"last_updated_date":1561633089932,"last_updated_date_str":"06/27/2019 04:15:09 PM",
"last_updated_by_name":null,"schedule_on":0,"paused_at_step":0,"percent_complete":0,
"job_action_type":null,"child_job_id":-1}
```

The response indicates that the generate workflow job is in queue.

Run the workflow

Run the workflow to create test data in the target connection.

You can run the `/tdm/v1/jobs` API to run a workflow.

1. Enter the following cURL API request:

```
curl -X POST "http://<host>:<port>/api/public/tdm/v1/jobs" -H "accept: application/json"
-H "authorization: Basic TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg==" -H "Content-
Type: multipart/form-data" -F "type=Execute" -F "projectId=2412" -F "planId=2211" -F
"integrationServiceName=PCIS"
```

2. Press Enter.
3. Review the response.
   The following code is the response that the sample request returns:

```
{"id":3420,"name":"Execute Workflow","description":"Plan: Plan_mask_test, Workflow:
","status":"In Queue",
"start_date":0,"end_date":0,"job_type":"Execute Workflow",
"created_by_name":null,"creation_date":1561633441828,
"creation_date_str":"06/27/2019 04:34:01 PM",
"last_updated_date":1561633441828,"last_updated_date_str":"06/27/2019 04:34:01 PM",
```

```
"last_updated_by_name":null,"schedule_on":0,"paused_at_step":0,"percent_complete":0,
"job_action_type":null,"child_job_id":-1}
```

The job to run the workflow has an ID of 3420. To verify that the workflow runs successfully, you can monitor the workflow execution job.

Monitor the Workflow Execution Job

You can monitor the status of the workflow execution job to confirm that it finishes successfully.

You can run the `/tdm/v1/jobs/{jobId}` API to view the status of a job.

1. Enter the following cURL API request:

```
curl -X GET "http://<host>:<port>/api/public/tdm/v1/jobs/3420" -H "accept: application/
json" -H "authorization: Basic TmF0aXZlL0FkbWluaXN0cmF0b3I6QWRtaW5pc3RyYXRvcg=="
```

2. Press Enter.
3. Review the response.
   The following code is the response that the sample request returns:

```
{"id":3420,"name":"Execute Workflow","description":"Plan: Plan_mask_test, Workflow:  ",
"status":"Succeeded","start_date":1561633289541,"end_date":1561633333497,
"job_type":"Execute Workflow",
"created_by_name":"Administrator","creation_date":
1561633222055,"creation_date_str":"06/27/2019 04:30:22 PM",
"last_updated_date":1561633222055,"last_updated_date_str":"06/27/2019 04:30:22 PM",
"last_updated_by_name":"Administrator","schedule_on":0,"paused_at_step":
0,"percent_complete":100,
"job_action_type":null,"child_job_id":-1}
```

The response indicates that the workflow execution job succeeded. You can now update and include the cURL requests in a script to automate these tasks.

Author

Sadhana Kamath

Acknowledgements

The author would like to acknowledge Nagaraju Battula and Manish KM for their technical assistance.